第三章 RISC-V 处理器设计和验证

图像处理 SoC 使用自研 RISC-V 处理器来完成控制和交互类任务，RISC-V 处理器是 SoC 上的重要 IP。本章主要介绍处理器的硬件设计和验证，首先介绍自研 RISC-V 处理器的微架构，包括分支预测、寄存器重命名、预取操作数、发射、加载与存储单元（LSU）、除法器、交付与退休、总线互联单元（BIU）、L1-数据缓存（L1-DCache），再介绍自研 RISC-V 处理器的测试，包括基于 riscv-tests 的指令集兼容性测试、coremark 跑分测试、嵌入式程序测试。

本自研 RISC-V 处理器已开源到 https://github.com/CutestPanda/Opensoc/tree/main/601_panda_risc_v_2，可从该网址获取相关源码和示例工程。

3.1 自研 RISC-V 处理器微架构

小胖达 RISC-V 是一款完全自研、使用方便的 32 位微控制器（MCU），支持 RV32IM，适用于嵌入式系统、显示驱动、中端计算等场景。该处理器是 6 级流水线、乱序单发射的，其结构如图 3-1 所示。它具有以下特点：

1. 动态分支预测，预测方向基于局部/全局分支历史，预测地址基于分支目标缓存（BTB）和返回地址堆栈（RAS）；支持乱序发射、乱序执行；寄存器重命名基于重排序队列（ROB）实现。
2. 支持精确的中断和异常，带有平台级中断控制器（PLIC）和局部中断控制器（CLINT），可支持多达 63 个外部中断源。
3. 带有数据缓存（DCache），支持以 DDR 作为主存。
4. 带有调试传输模块（DTM）和调试模块（DM），支持 JTAG 烧录下载和外部在线调试。
5. 带有符合 AXI-Lite 协议的指令、数据、外设总线，可连接不同类型的指令、数据存储器，可拓展各种外设，易于集成在片上系统（SoC）上。
6. 完全支持使用 Eclipse 开源 IDE 进行软件开发，可直接在 Eclipse 上编译、下载和调试 C 程序。
7. 性能对标 ARM Cortex-M3，跑分可达 3.188 Coremark/MHz。
8. 提供丰富的软件开发包（SDK），包括外设驱动（如 GPIO、UART、SPI、I2C 等）、中断控制器驱动（如 PLIC、CLINT）、重定向的标准输入输出函数、延时函数、DCache 控制函数等。

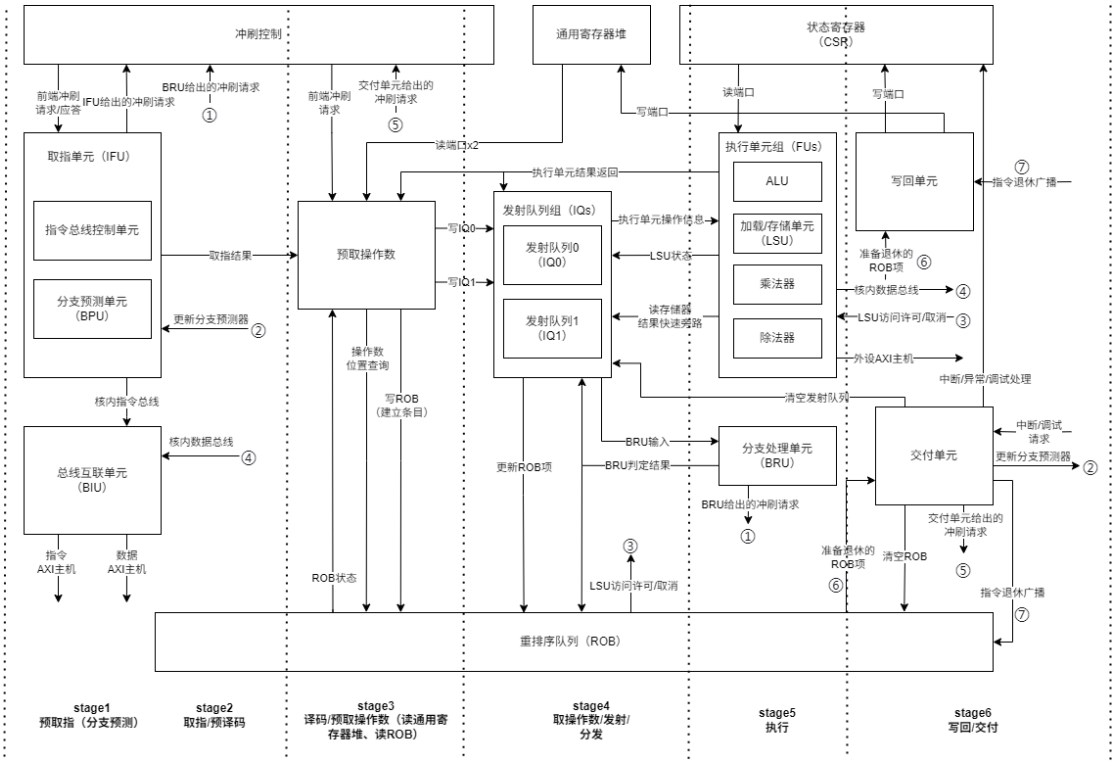


图 3-1 RISC-V 处理器核流水线图

3.1.1 分支预测单元

RV32I 定义了 3 种分支指令，分别是无条件直接跳转指令（JAL 指令）、无条件间接跳转指令（JALR 指令）和有条件直接跳转指令（B 指令），直接跳转指令的目标地址在编译时就已确定（通常由 PC+立即数得到），而间接跳转指令的目标地址则需要在运行时才能确定（比如由寄存器值+立即数得到）。

由于 JALR 指令的目标地址、B 指令是否跳转都要在取得操作数后才能确定，而取指应当是连续不断的，因此需要由分支预测单元（BPU）提前向取指单元（IFU）给出下一待取指令的地址。对于深度流水线的 CPU 来说，分支预测失败导致的惩罚是较大的，包括流水线的冲刷、逻辑寄存器映射表（RAT）的恢复等额外操作，会产生若干个到十几个周期的空泡，因此提高分支预测的准确率是提升 CPU 性能的关键。

在小胖达 RISC-V 的 BPU 中，分支目标缓存（BTB）记录了分支指令的类型、返回地址堆栈（RAS）出栈与入栈标志、分支目标地址（BTA），其本质是存储分支指令信息的缓存（Cache）；返回地址堆栈（RAS）主要用于预测 JALR 指令的目标地址；基于局部/全局分支历史的预测器使用分支历史表（BHT）来记录局部分支历史，使用全局历史寄存器（GHR）来记录全局分支历史，结合局部和全局分支

历史，由 PC 低位索引某个模式历史表（PHT），得到其中的两比特饱和计数器，用于预测 B 指令的跳转方向。

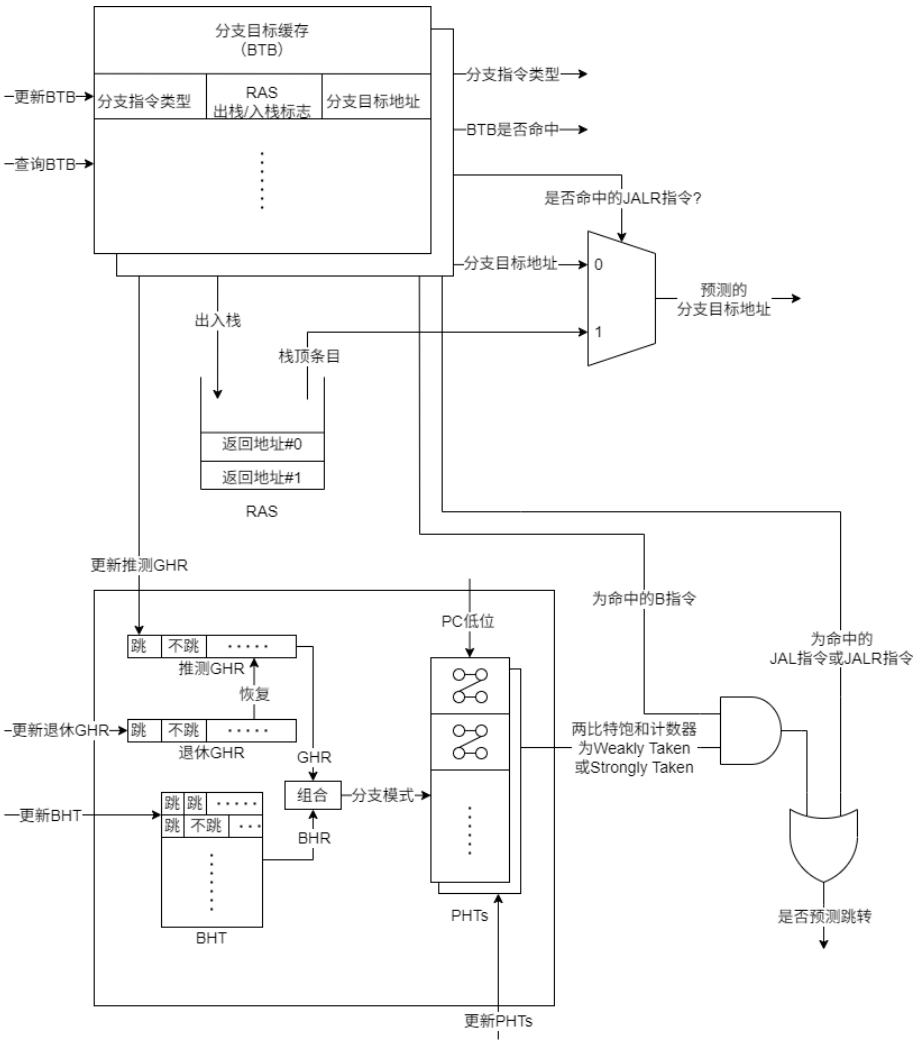


图 3-2 小胖达 RISC-V 的 BPU 设计

特别地，当 BTB 命中了一条 RAS 出栈标志无效的 JALR 指令时，JALR 指令的 BTA 无法从 RAS 中得到，这在 switch-case 语句中比较常见。文献[X]提到了利用基于局部历史的分支预测方法中使用的 BHR 对目标地址进行预测，只是将 PHT 换成 BTA 缓存（Target Cache）即可。实际上，switch-case 语句可能的跳转地址是有限的，根据 JALR 指令以前的跳转地址情况，由 PC 低位索引某个 Target Cache，可以很好地预测这条 JALR 指令的目标地址。简单起见，小胖达 RISC-V 并未实现这种预测，在遇到这种情况时，阻塞 BPU 并等待后级流水线返回真实的分支指令类型和 BTA。

测得小胖达 RISC-V 的分支预测准确率如表 3-1 所示。

表 3-1 小胖达 RISC-V 的分支预测准确率

分支指令类型	预测方式	预测准确率
JAL 指令	由 BTB 预测跳转地址	99.946%
JALR 指令	由 RAS 预测跳转地址	97.597%
B 指令	由 BTB 预测跳转地址，由基于局部/全局分支历史的预测器预测方向	94.479%

分支预测准确率是在运行 coremark 程序时测得的。测试时 BPU 的配置参数：BHT 深度 ＝512、BHT 位宽 ＝13、PHT 地址位宽 ＝16、未使用 GHR；BTB 路数 ＝2、BTB 深度 ＝1024；RAS 深度 ＝4。另测得 BTB 命中率为 99.973%。

3.1.2 寄存器重命名

小胖达 RISC-V 使用基于重排序队列（ROB）的寄存器重命名来解决伪数据相关性，包括先写后写（WAW）和先读后写（WAR）相关性。在乱序发射 CPU 中，后序指令可能比前序指令先发射和执行，如果没有寄存器重命名，则后序指令的结果先写回逻辑寄存器堆（ARF）会导致相关逻辑寄存器的最终值不是最新的，或者导致前序指令读到错误的操作数。

代码段 3-1 WAW 相关性问题

```
mul x3, x1, x2
add x3, x6, x7 ;add 指令可能比 mul 指令先发射和执行，导致 x3 的最终值是 mul 指令
               ;的结果而非 add 指令的结果
```

代码段 3-2 WAR 相关性问题

```
div x1, x3, x2
xor x3, x6, x7 ;xor 指令可能比 div 指令先发射和执行，导致 div 指令错误地读取到 xor 指令
               ;的结果作为操作数 1
```

ROB 维护了指令原有的顺序，如果给 ROB 配备一个用于暂存执行单元（FU）结果的缓存区，即使 FU 的结果是乱序返回的，ROB 也能按顺序将暂存结果写回 ARF。ROB 还维护了一张逻辑寄存器映射表，记录了每个逻辑寄存器值是位于 ARF、ROB 的结果缓存区还是旁路网络上，以保证取操作数的正确性，包括从 ARF 或 ROB 的结果缓存区预取操作数，以及在给定 FU 上捕获执行结果。因此，只要能够给指令在 ROB 的结果缓存区里分配 1 项，那么 WAW 和 WAR 相关性就已经被解决。而针对先写后读（RAW）的真数据相关，则是由逻辑寄存器映射表来跟踪操作数的位置。

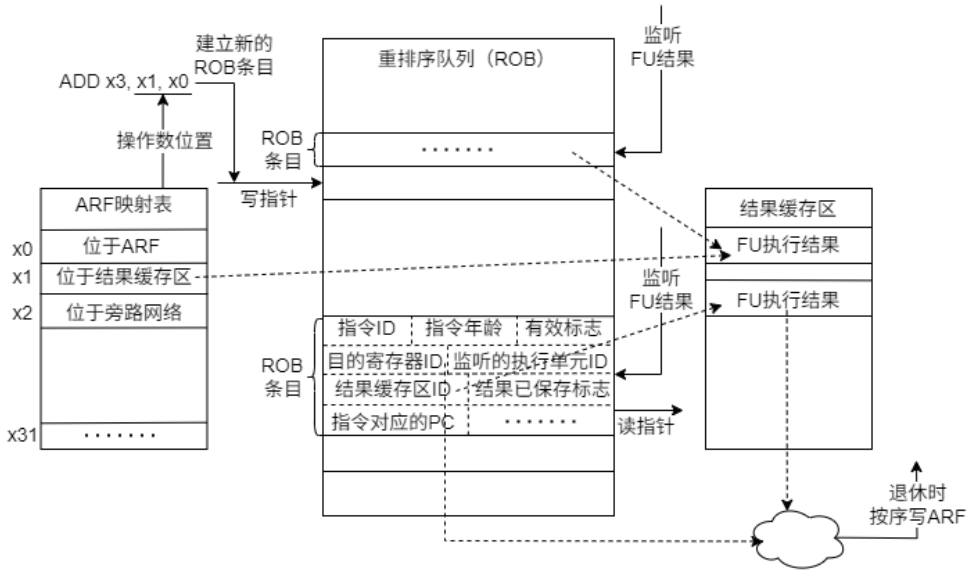


图 3-3 基于 ROB 的寄存器重命名

3.1.3 预取操作数与乱序发射

小胖达 RISC-V 采取了数据捕获的发射策略，也就是说，在发射之前先有 1 级预取操作数的流水线，这级流水线会读取 ARF 和 ROB 里暂存的结果，若操作数尚未就绪，则预取操作数处的段寄存器、发射队列（IQ）里的操作数字段会从 FU 的结果输出捕获数据。对于小胖达 RISC-V 来说，IQ 不会太深，数据捕获所增加的操作数字段并不会占有很大的面积。并且，由于读 ARF 和 ROB 暂存结果是在发射之前进行的，又因为 ROB 里记录的是已进入 IQ 或已发射的指令，ROB 到 ARF 的数据搬运并不会影响到后级取操作数，后级仅需捕获 FU 的结果即可，因此数据捕获的发射策略是匹配基于 ROB 的寄存器重命名的。

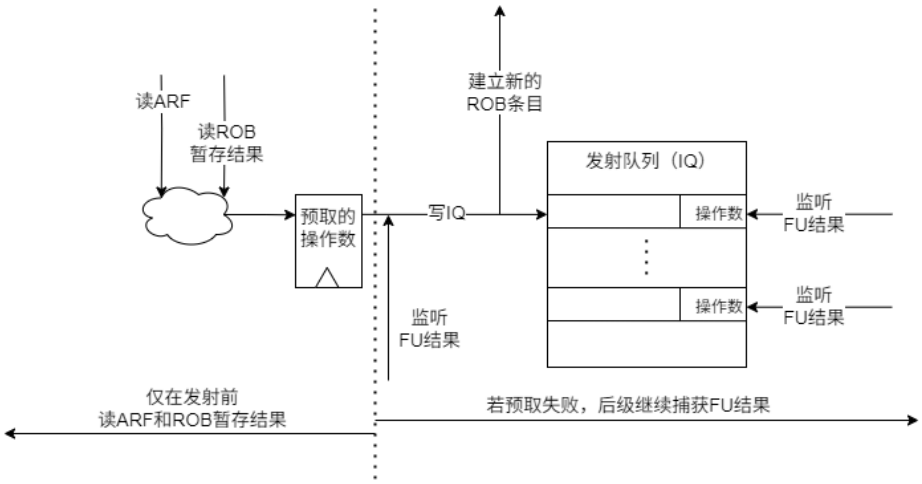


图 3-4 数据捕获的发射策略

小胖达 RISC-V 具有 2 个发射队列。一个发射队列是可以乱序发射的，接收算术逻辑指令、CSR 读写指令、乘除法指令、特权指令、非法指令；该发射队列是非压缩的，写入时选择编号最小的无效项，发射时选择最旧的有效项。另外一个发射队列只能进行顺序发射，接收分支指令、内存屏障指令、加载与存储指令、断点指令。两个发射队列都只能在分支路径已确认时接收新的指令，若不然，在发生分支预测失败时需要对逻辑寄存器映射表进行恢复，并取消错误分支路径下的所有指令，这对于 MCU 级别的小胖达 RISC-V 来说稍显复杂。将第 2 个发射队列设为顺序发射是经过以下考量的：

1. 断点指令没有作乱序发射的必要，因为在线调试对性能要求并不高。

2. 内存屏障指令通常要求前序访存指令已产生效果后才能发射，这样苛刻的条件导致其乱序发射并不会带来有效的性能提升。

3. 尽管允许后序分支指令先发射能提高处理器的性能，但实现这一功能需要较多的检查点以在分支预测失败时对处理器状态进行回滚。而且，最旧的分支指令更应当尽快发射，以确认较旧指令是否处于正确的分支路径上。虽然小胖达 RISC-V 没有实现分支指令的乱序发射，但是它支持分支指令的乱序确认，只要分支指令的操作数就绪，后序分支指令可以先确认。

4. 加载与存储指令的乱序发射涉及到加载与存储地址的相关性检查，这会使得加载与存储单元（LSU）的设计变得复杂。

特别地，由于加载（Load）指令经常是处于数据相关性的顶端，为了尽快确认分支路径和“唤醒”发射队列中的条目，应当对加载指令的执行结果作快速旁路。

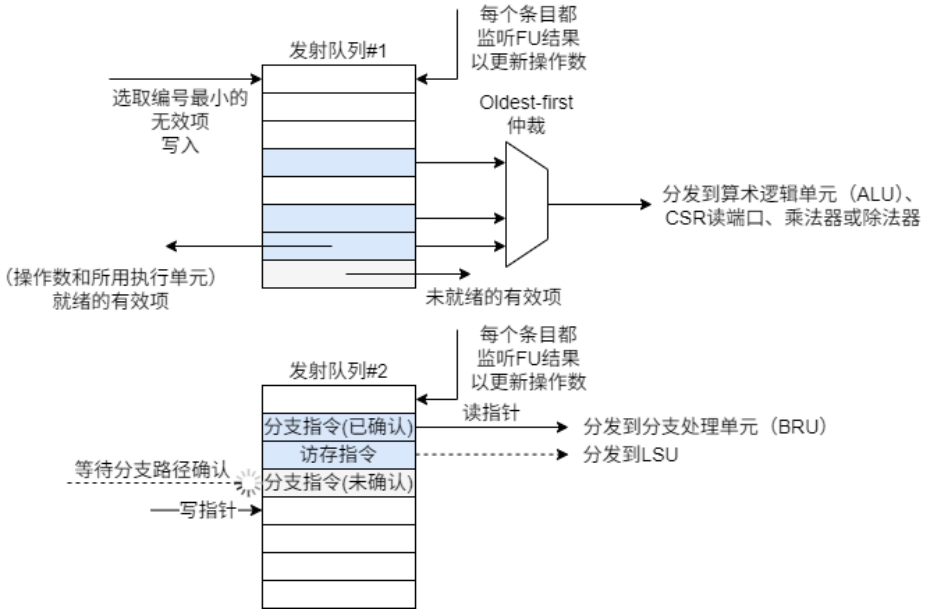


图 3-5 小胖达 RISC-V 的发射队列设计

3.1.4 加载与存储单元

LSU 是一种用于处理加载或存储指令的执行单元。LSU 接受包含访存对齐方式、访存地址和写数据等信息的访存请求，在数据总线或外设总线上发起事务，返回包含读数据、错误码等信息的访存响应。LSU 处理的访存请求可分为 3 类：

1. 写存储器请求。LSU 在接受该请求时就将其视为已完成，并给出访存响应。与此同时，该请求会被置入写存储器请求缓存区，等待来自 ROB 的访存许可后才能在数据总线上发起写事务。更确切的说，一条写存储器指令只有在它成为 ROB 里最旧的指令时才被允许启动总线写事务，比如写 DCache，否则，当前序指令产生异常或被标记为中断并导致这条写存储器指令需要被取消时，它所提前执行的写 DCache 操作无法被撤销。尽管写存储器的执行有所延后，写存储器请求缓存区的存在也使得 LSU 可以继续接受后序的存储器访问请求，从而提高效率。图 3-6 展示了写存储器请求缓存区中 2 个由寄存器组构成的表，请求信息表中有效且取得访存许可的条目会被载入事务记录表，确切地说，它被合并到事务记录表里的某个有效条目，或者被存入到 1 个新条目。该图给出了处理写存储器请求的步骤，然而，2 个表中众多条目所处的处理阶段可以是不同的，因此对写存储器请求的处理是非阻塞、可滞外的。

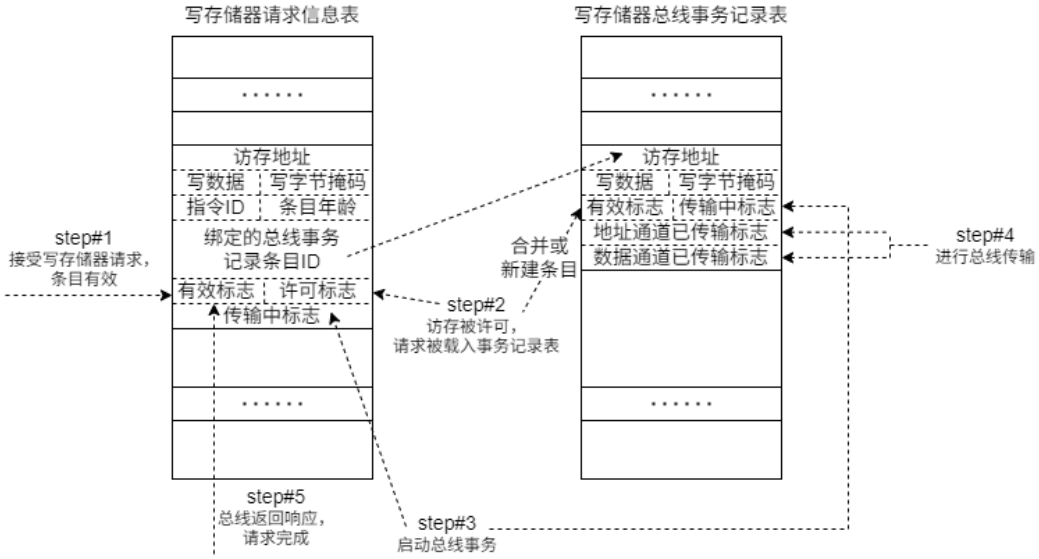


图 3-6 LSU 的写存储器请求缓存区

2. 读存储器请求。LSU 在接受该请求时可以立刻在数据总线上发起读事务。这是因为，读存储器的执行与否并不会影响 CPU 的状态。另外，在接受该请求时还需要检查写存储器请求信息表，以便在数据 AXI 主机的读通道返回响应时能合并上最新的修改数据及其字节掩码。图 3-7 展示了如何在 8 个条目的写存储器请求

信息表里找到某个访问地址相匹配的字节的最新修改记录。

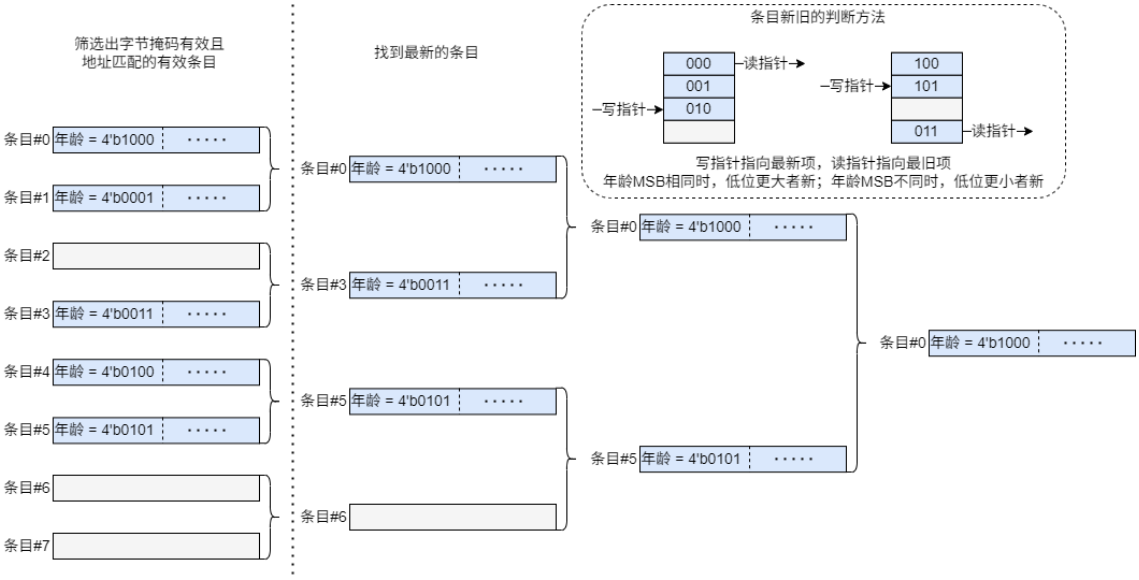


图 3-7 检查写存储器请求信息表以得到最新的修改记录

3. 外设访问请求。与写存储器请求类似，该请求也要等待来自 ROB 的访存许可后才能在外设总线上发起事务。这是因为，外设访问可能带有副效应，比如访问某个带有读清零字段的寄存器时。另外，考虑到当 1 次外设访问未完成前，无法确定它是否发生总线错误，小胖达 RISC-V 的 LSU 没有对外设访问实现滞外传输（Outstanding）的特性，也就是说，后序的外设访问必须等待前序的外设访问完成后才能发起事务。

3.1.5 除法器

非迭代除法器主要是基于恢复余数法、不恢复余数法和 SRT 这 3 种算法。恢复余数法最为简单，但它带有冗余操作，即试商失败时需要加回除数，这会增加组合逻辑延迟。不恢复余数法则没有恢复余数的步骤，而仅需额外 1 个周期来修正商和余数。SRT 在这三者中性能最高，但也是最复杂、面积最大的，它需要经过除数归一化、商迭代、商转换、修正余数等步骤。另外，迭代除法器主要是基于牛顿迭代法或Goldschmidt算法，这种除法器计算速度更快，但它得到的商是有误差的，因此一般在 GPU 或 AI 加速器等低精度、高性能的计算场景下应用较多。

考虑到复杂度、性能、面积的折中，小胖达 RISC-V 的除法器采用了不恢复余数法来实现的。该除法器共需 36 个周期来完成 33 位的有符号除法，表 3-2 描述了各个周期完成的逻辑。

表 3-2 除法器各个周期完成的逻辑

周期	完成的逻辑	说明
1	预置阶段（载入被除数） 除零与溢出判断	若发生除零或溢出，则后续阶段均被跳过
2~34	计算阶段	每个周期完成 1bit 上商和 1 次余数更新
35	修正商与余数	若不需要作修正，则本周期可跳过
36	进行整除修正	

除法器的结构如图 3-8 所示。设置商寄存器 Q、余数寄存器 R、被除数寄存器 X、除数寄存器 Y，在预置阶段将符号位拓展后的 X 载入 R 和 Q，在计算阶段的每个周期里根据 R 的符号和 Y 的符号完成上商、更新 R、左移 R 和 Q 的操作，注意在最后一次计算时仅左移 Q，最后可能需要修正 Q 和 R。

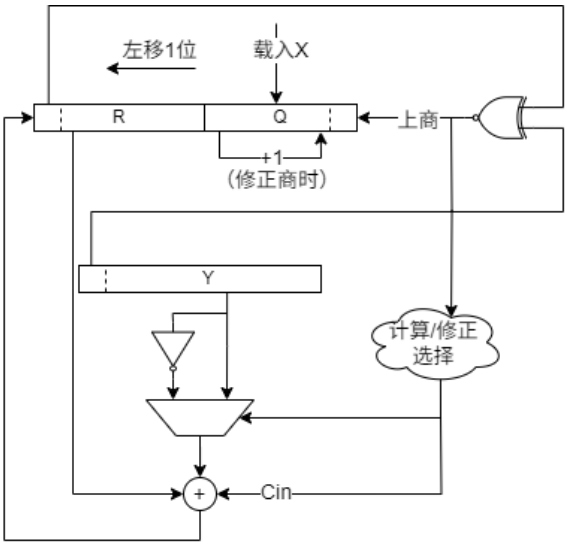


图 3-8 除法器结构

实际上，小胖达 RISC-V 的除法器也支持在计算阶段的每个周期完成 2bit 上商和 2 次余数更新，从而将计算阶段消耗的周期数减少为 17 个，但这会增加组合逻辑的延迟。

3.1.6 交付与退休

交付与退休处于指令流水线的末端，这一阶段是按照原有的指令顺序进行的。

当一条指令成为 ROB 里最旧的指令并且已经取得相应 FU 的执行结果时，如果它不带有同步异常，那么其结果将被写入指定的通用寄存器（GPR）或状态寄存

器（CSR），从而对 CPU 状态产生影响，这就是指令的退休。

然而，某些指令可能会发生例外，这包括指令本身带有的同步异常，也包括指令所绑定的中断或调试请求。因此，在指令退休前还需要有一个交付阶段，该阶段会对异常、中断、调试请求进行仲裁，确定所要处理的例外类型，修改相关的 CSR，将指令流水线冲刷到异常或调试服务函数的入口。表 3-3 列出了小胖达 RISC-V 所处理的异常、中断和调试请求。除此之外，例外返回指令如 MRET 或 DRET 会使指令流水线冲刷到例外返回地址。为了方便软件作异常处理，小胖达 RISC-V 的所有同步异常都能精确到某条指令。

表 3-3 小胖达 RISC-V 所处理的异常、中断和调试请求

例外类型	例外原因
同步异常	指令地址非对齐 指令总线访问错误 非法指令 读存储映射地址非对齐 读存储映射总线错误 写存储映射地址非对齐 写存储映射总线错误 机器模式环境调用
中断	机器模式软件中断 机器模式计时器中断 机器模式外部中断
调试请求	断点指令导致的调试请求 调试器暂停请求 单步调试 复位释放后暂停请求

3.1.7 总线互联单元

总线互联单元（BIU）是一个两主两从的交叉开关矩阵（Crossbar）。核内指令总线是只读的，它只能访问外部指令总线。核内数据总线是可读可写的，它既能访问外部数据总线，也能访问外部数据总线。核内总线与外部总线的互联关系如图 3-9 所示。之所以这样设计，是因为核内指令总线只用于取指令，而核内数据总线需要访问存储映射的数据和指令存储器区域，具体包括访问数据缓存（DCache）、读指令存储器里的常量数据、烧录代码程序等操作。可见，BIU 并非完全的交叉开关矩阵，某些访问路径比如从核内指令总线到外部数据总线的路径是不存在的，专门设计这样的 BIU 而非使用通用的总线互联矩阵可以减小一些面积。

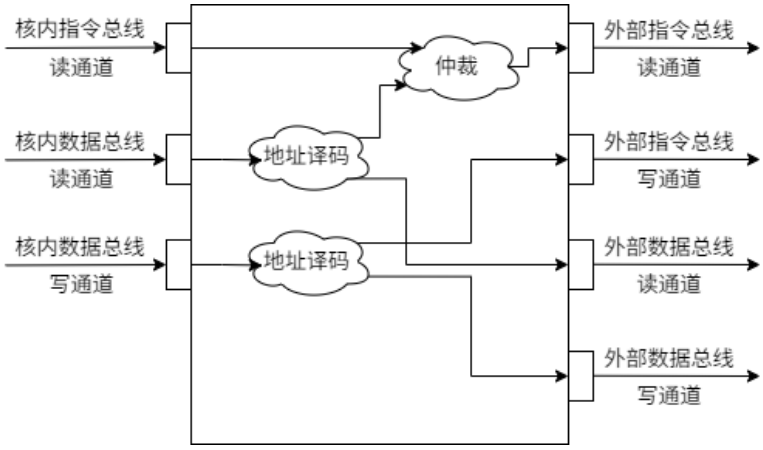


图 3-9 核内总线与外部总线的互联

图中箭头代表总线的主从关系，而非数据流的流向。箭头始端为主机，箭头末端为从机。

3.1.8 L1-数据缓存

为了降低CPU访问主存的时延，通常会在CPU与主存之间插入缓存（Cache），否则，主存上百个周期的访问时延对于 CPU 来说是不可容忍的。Cache 能够降低访存时延的原理是 CPU 对数据的访问具有时间局部性和空间局部性，也就是说，如果某个数据被访问，那么接下来一段时间它很可能还会被访问，并且它周围的数据在以后也可能被访问。考虑到成本和性能的折中，现代处理器通常采用 3 级缓存，L1 缓存速度快、容量小，而 L2、L3 缓存速度慢、容量大，这样一来，即使 L1 缓存缺失，CPU 也可以先访问具有几十个周期时延的 L2、L3 缓存，而非低速的主存。

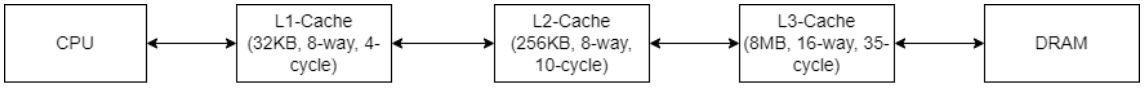


图 3-10 多级 Cache 结构

小胖达 RISC-V 实现了私有的 L1-DCache。该缓存是组相连、写回、写分配的，且带有写缓存（Write Buffer）。L1-DCache 的控制逻辑主要围绕三个静态存储器（SRAM）展开，数据存储器（Data Sram）用于存储数据块，标签存储器（Tag Sram）用于保存缓存行的地址标签和标志，热度存储器（Hot Sram）记录了缓存集里各个缓存行的热度，具体功能如图 3-11 所示。

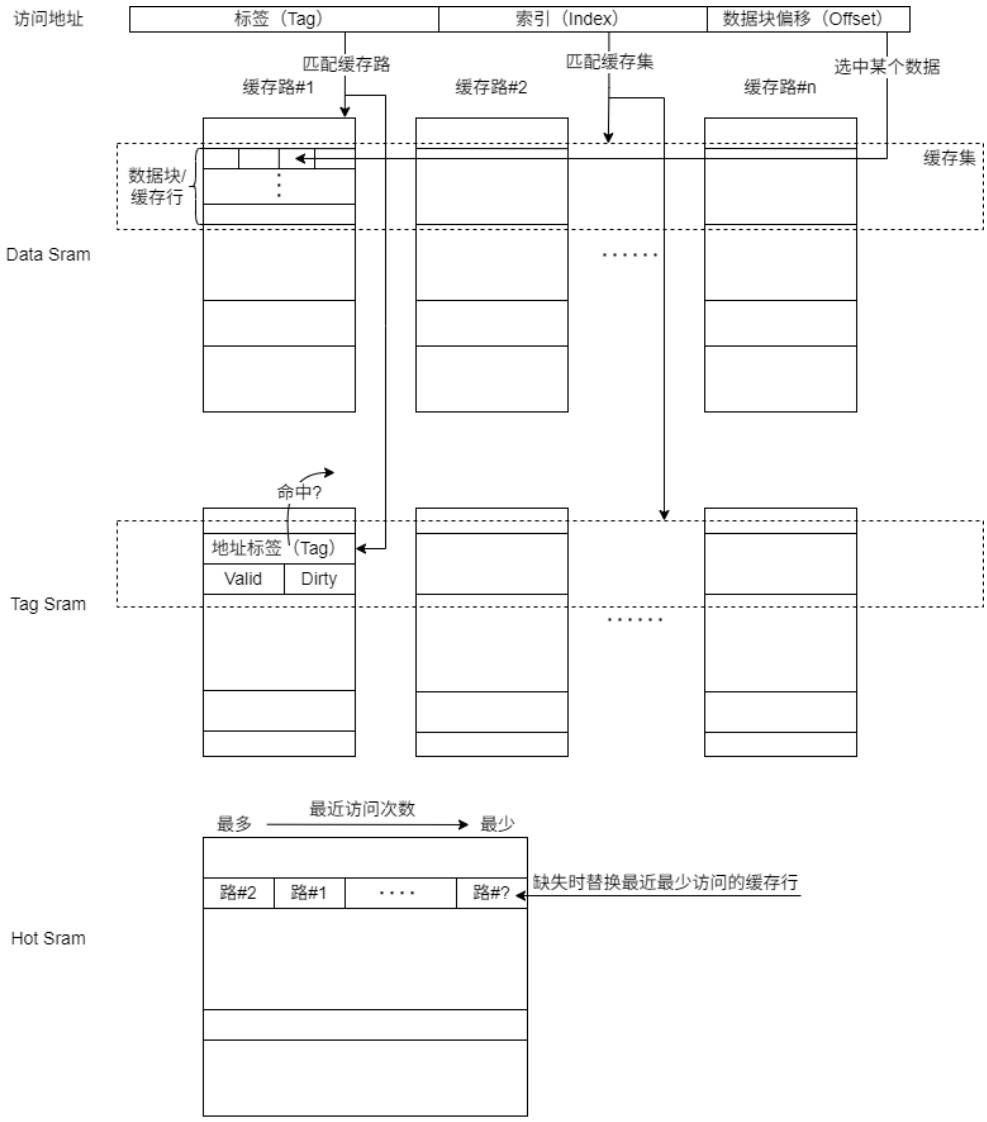


图 3-11 L1-DCache 里的 Data Sram、Tag Sram 和 Hot Sram

L1-DCache 的工作流程如图 3-12 所示，可分为读命中、写命中、读缺失、写缺失这 4 种情况，若命中，则访问时延为 1 个时钟周期，否则需要读下级存储器并将脏的待替换缓存行送入写缓存。

设置写缓存的目的是隐藏将脏的待替换缓存行写入下级存储器所用的时间，送入写缓存的数据将在总线空闲时启动写事务。需要注意的是，Cache 缺失时，写缓存里的数据块比下级存储器的更新，因此必须先检索写缓存再发起读下级存储器的请求。

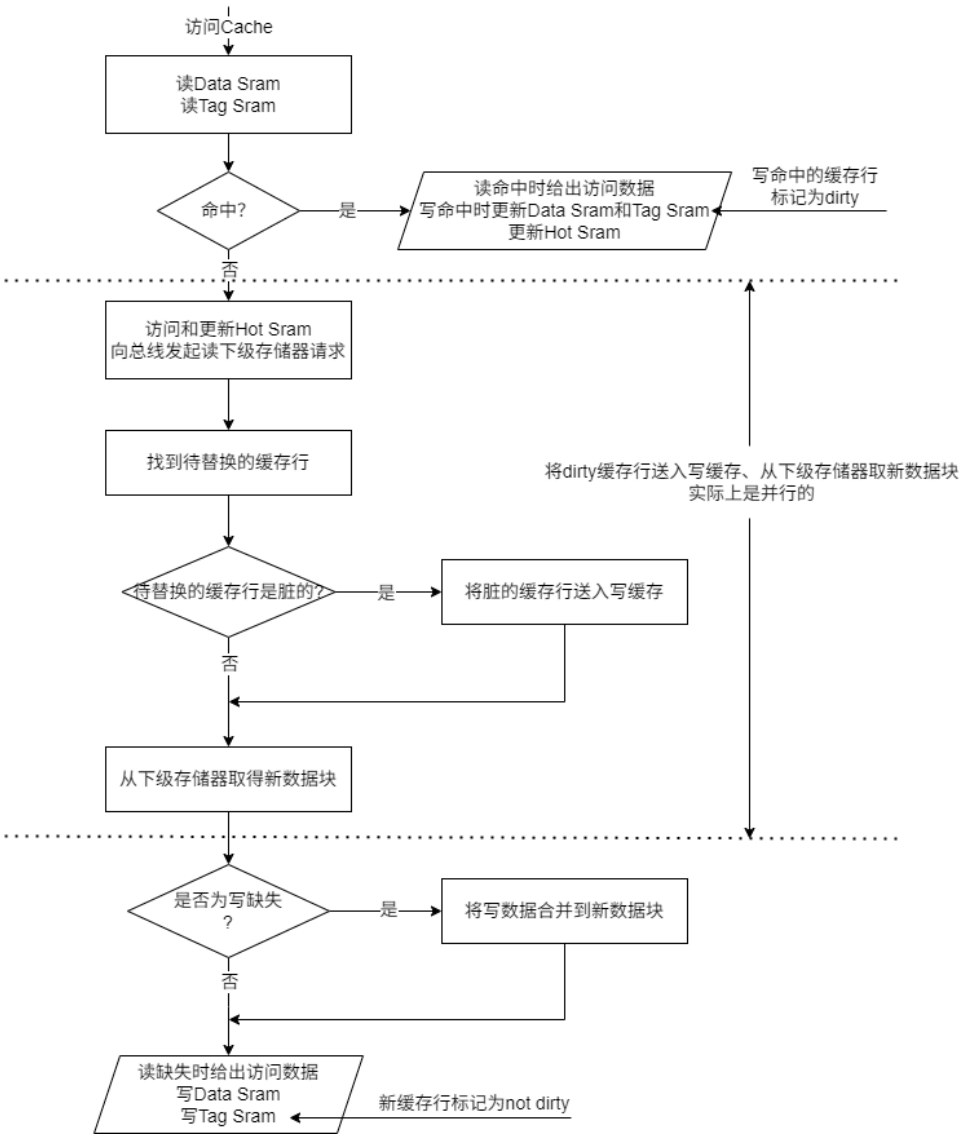


图 3-12 L1-DCache 的工作流程

在小胖达 RISC-V 上运行一段对 720p 灰度图像作 3x3 均值滤波的程序，测得 L1-DCache 的命中率如表 3-4 所示。

代码段 3-3 L1-DCache 测试程序

```
uint8_t img_org[1280 * 720];
uint8_t img_res[1280 * 720];

for(int i = 0;i < 720;i++){
    for(int j = 0;j < 1280;j++){
        if(i != 0 && i != 719 && j != 0 && j != 1279){ //  不处于图像边界
            uint16_t sum = 0;
```

代码段 3-3 L1-DCache 测试程序（续）

```
                // 对原图像 3x3 内的 ROI 求和（省略该段代码）…

                img_res[i * 1280 + j] = sum / 9;
            }else{ // 处于图像边界
                img_res[i * 1280 + j] = img_org[i * 1280 + j];
            }
        }
}
```

表 3-4 L1-DCache 命中率测试结果

Cache 路数	Cache 容量	数据块大小	命中率
4	32KB	16B	99.715%
		32B	99.858%
	16KB	16B	99.715%
		32B	99.858%

3.2 自研 RISC-V 处理器的测试

RISC-V 处理器的验证通常包括以下几步：

1. 模块级验证。对某些独立模块如算术逻辑单元（ALU）、取指单元（IFU）、加载与存储单元（LSU）进行功能验证，这一步通常是进行基于通用验证方法学（UVM）的定向测试与随机测试，并插入 System Verilog 断言（SVA）作协议检查。

2. 处理器核验证。这一步使用指令生成器如 riscv-dv 生成随机指令序列，使用模拟器如 spike 作为参考模型，运行程序并对比参考模型和待测核的指令跟踪记录（Trace）以确定指令执行路径的一致性。

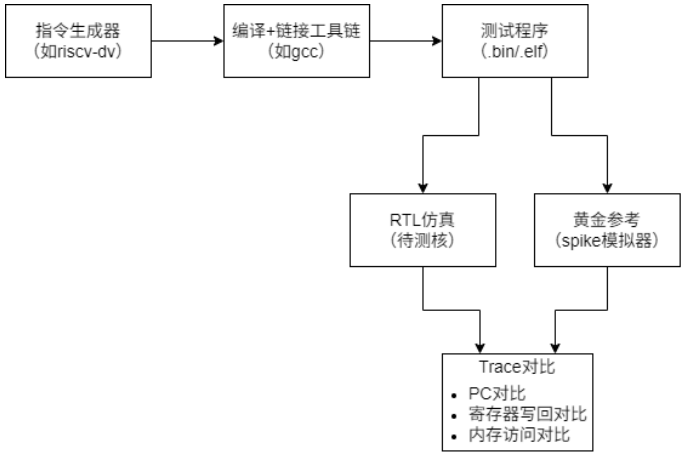


图 3-13 RISC-V 处理器核验证框图

3. SoC 级验证。将处理器核集成到片上系统，验证包括多核缓存一致性如 MESI 或 MOESI 协议、片上互联如 AXI 或 CHI 总线、外设访问如 DDR 等系统功能。通常地，大型 SoC 验证会采用硬件仿真（Emulation）或 FPGA 来加速进程。

在仿真验证阶段，本文对自研 RISC-V 处理器完成了基于 riscv-tests 的指令集兼容性测试，这相当于处理器核级别的定向测试。在 FPGA 原型验证阶段，本文在自研 RISC-V 处理器上作了 coremark 基准测试，还将处理器集成到小型 SoC 上运行了嵌入式软件程序。

3.2.1 基于 riscv-tests 的指令集兼容性测试

riscv-tests 是 RISC-V 官方维护的开源测试套件，它是手写汇编测试，用于验证 RISC-V 处理器是否符合指令集（ISA）规范。与 riscv-dv 生成的随机程序不同，riscv-tests 是定向测试，不过，riscv-tests 无需像 spike 那样的参考模型，因此使用起来更加简单，它可以作为处理器核基本而快速的验证。

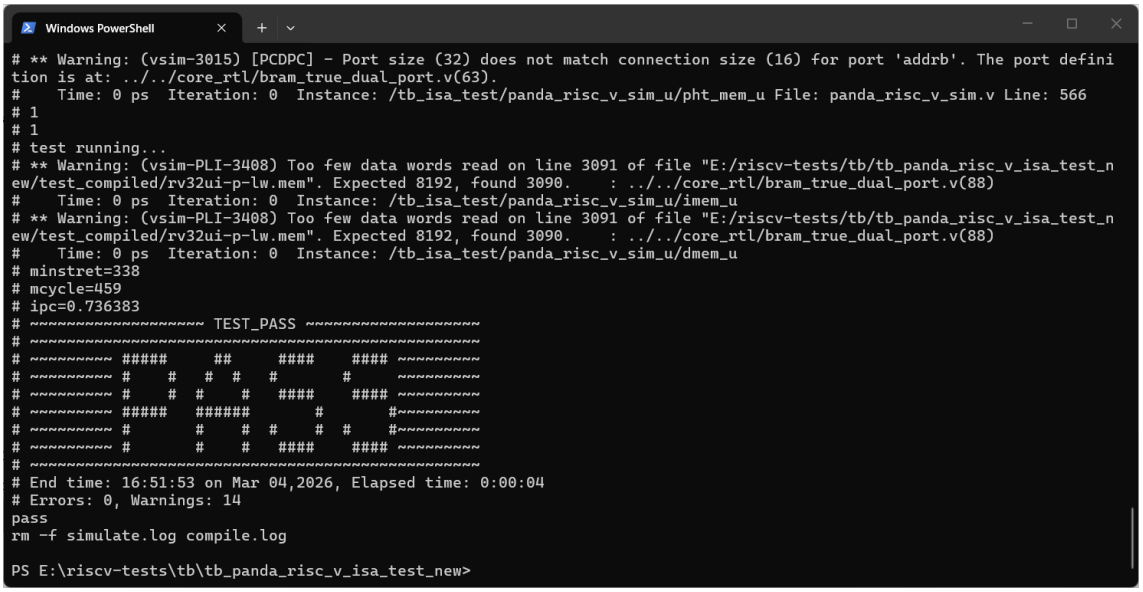
riscv-tests 里的每个测试都是独立的汇编文件。基于 riscv-tests 的 ISA 兼容性测试的步骤如下：

1. 编写链接脚本。本文所采用的链接方式如表 3-5 所示。
2. 使用 GCC 工具链将汇编程序转换为机器码。
3. 编写测试平台（TB）。在 TB 里，将机器码导入指令和数据存储器，等待 CPU 向测试输出地址写数据后抓取结果。

表 3-5 开展基于 riscv-tests 的 ISA 兼容性测试时采用的链接方式

段	基地址	内容
.text.init	0x00000000	初始化代码段
.text	对齐到下一个 4KB 地址	主代码段
.data	对齐到下一个 4KB 地址	已初始化数据段
.bss	紧邻上一段	未初始化数据段
.tohost	0x00003000	测试输出

对于 rv32ui-p 测试集，除了名为 rv32ui-p-ma_data 的非对齐访存测试，小胖达 RISC-V 能够通过其余 41 个测试。对于 rv32um-p 测试集，小胖达 RISC-V 能够通过全部 8 个测试。例如，图 3-14 是通过名为 rv32ui-p-lw 的加载指令（load 指令）测试时的结果输出。



```
Windows PowerShell
# ** Warning: (vsim-3015) [PCDPC] - Port size (32) does not match connection size (16) for port 'addrb'. The port defini
tion is at: ../../core_rtl/bram_true_dual_port.v(63).
#    Time: 0 ps  Iteration: 0  Instance: /tb_isa_test/panda_risc_v_sim_u/pht_mem_u File: panda_risc_v_sim.v Line: 566
# 1
# 1
# test running...
# ** Warning: (vsim-PLI-3408) Too few data words read on line 3091 of file "E:/riscv-tests/tb/tb_panda_risc_v_isa_test_n
ew/test_compiled/rv32ui-p-lw.mem". Expected 8192, found 3090.    : ../../core_rtl/bram_true_dual_port.v(88)
#    Time: 0 ps  Iteration: 0  Instance: /tb_isa_test/panda_risc_v_sim_u/imem_u
# ** Warning: (vsim-PLI-3408) Too few data words read on line 3091 of file "E:/riscv-tests/tb/tb_panda_risc_v_isa_test_n
ew/test_compiled/rv32ui-p-lw.mem". Expected 8192, found 3090.    : ../../core_rtl/bram_true_dual_port.v(88)
#    Time: 0 ps  Iteration: 0  Instance: /tb_isa_test/panda_risc_v_sim_u/dmem_u
# minstret=338
# mcycle=459
# ipc=0.736383
# ~~~~~~~~~~~~~~~~~~~~ TEST_PASS ~~~~~~~~~~~~~~~~~~~~
# ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# ~~~~~~~~~~ #####      ##      ####     #### ~~~~~~~~~~
# ~~~~~~~~~~ #    #   #  #    #       #      ~~~~~~~~~~
# ~~~~~~~~~~ #    #  #    #   ####     #### ~~~~~~~~~~
# ~~~~~~~~~~ #####   ######       #        #~~~~~~~~~~
# ~~~~~~~~~~ #       #    #  #    #  #     #~~~~~~~~~~
# ~~~~~~~~~~ #       #    #   ####     #### ~~~~~~~~~~
# ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# End time: 16:51:53 on Mar 04,2026, Elapsed time: 0:00:04
# Errors: 0, Warnings: 14
pass
rm -f simulate.log compile.log

PS E:\riscv-tests\tb\tb_panda_risc_v_isa_test_new>
```

图 3-14 小胖达 RISC-V 通过名为 rv32ui-p-lw 的加载指令（load 指令）测试

图 3-15 展示了进行 riscv-tests 测试时一条加载指令（load 指令）从取指到写回的过程。其中，（a）时刻取到了这条加载指令，其汇编表示为 lw x14, 0(x2)，指令 ID 为 26，指令地址为 0x000001a8；（b）时刻该指令被写入发射队列#1；（c）时刻该指令所对应的访存请求被 LSU 所接受；（d）时刻 LSU 给出访存响应；（e）时刻指令的执行结果被写回通用寄存器 x14。

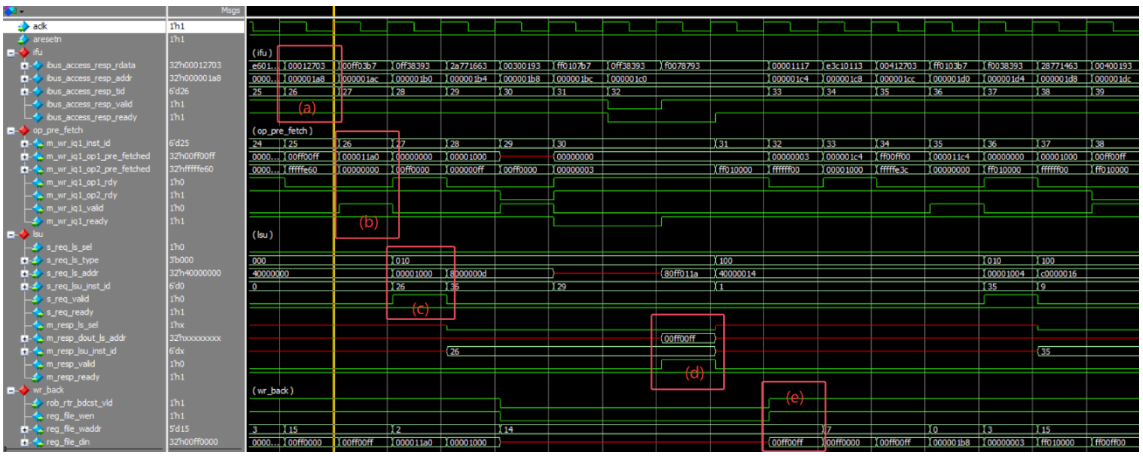


图 3-15 一条加载指令从取指到写回的过程的仿真波形图

3.2.2 Coremark 跑分测试

Coremark 是一种广泛使用的嵌入式处理器基准测试程序，用于测量处理器核的性能。与 Dhrystone 基准测试不同，Coremark 具有很强的抗编译器优化能力，因

此能够更好地测试处理器核本身的性能而不受编译器优化的影响。并且，Coremark 的测试内容更加丰富，它更能够体现真实的嵌入式工作负载，具体包括：

1. 矩阵运算，主要测试乘累加和数学运算能力。
2. 链表操作，主要测试指针使用和内存访问。
3. 状态机，主要测试数据依赖处理和分支预测能力。
4. 循环冗余校验（CRC），模拟嵌入式场景的常见任务，同时作为自验证机制。

Coremark 测试程序的移植很简单，只需要实现 core_portme.c 里的计时函数、时间单位转换函数和串口打印函数，并根据测试需要来修改 core_portme.h 里的宏定义即可。

在小胖达 RISC-V 上运行 Coremark 测试程序，串口终端的输出如图 3-16 所示。

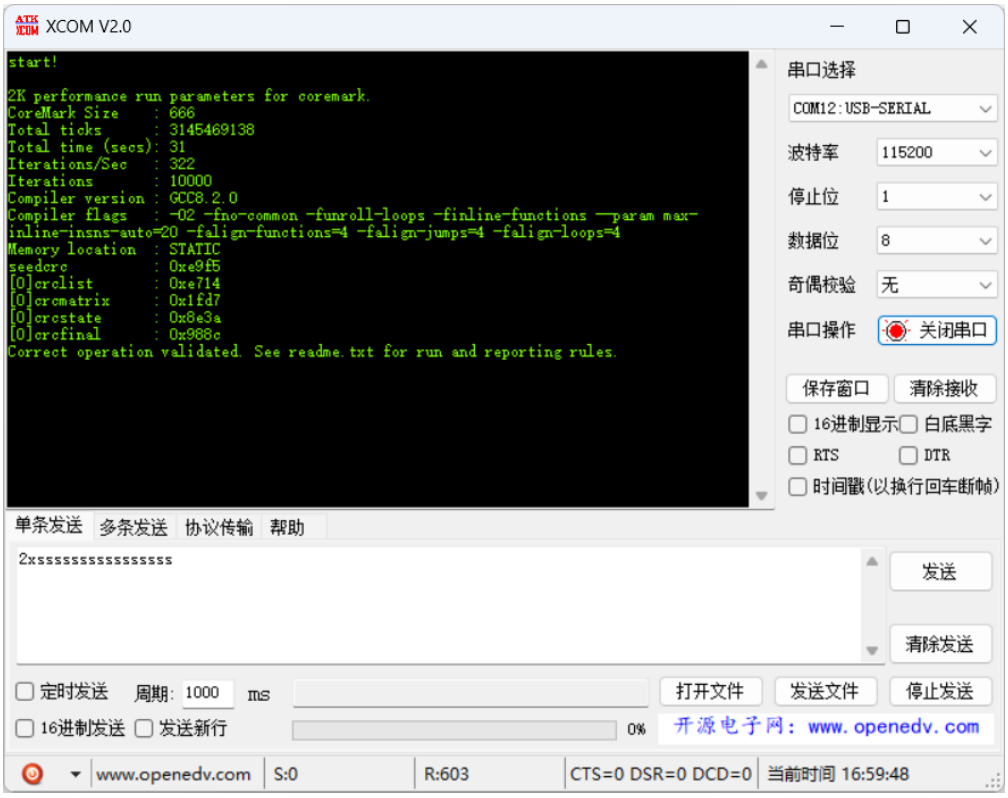


图 3-16 小胖达 RISC-V 运行 Coremark 测试程序时的串口终端输出

在本次测试中，总迭代次数为 10000 次，运行周期数为 3145469138，运行时钟频率为 100MHz，则小胖达 RISC-V 处理器的 Coremark 跑分为：

$$\frac{\text{每秒迭代次数}}{\text{运行时钟频率}}=\frac{\left(\frac{\text{总迭代次数}}{\text{运行时间}}\right)}{\text{运行时钟频率}}=\frac{\left(\frac{10000}{31.45469138}\right)}{100}\approx 3.1792\ \text{Coremark/MHz} \tag{3-1}$$

表 3-6 展示了小胖达 RISC-V 处理器与 STM32 系列处理器的 Coremark 跑分对比。另测得小胖达 RISC-V 处理器在运行 Coremark 测试程序时的每周期退休指令数（IPC）为 0.890，这基本上已经接近单发射乱序核的性能极限。

表 3-6 Coremark 跑分对比

处理器核	指令存储器类型	数据存储器类型	运行频率（MHz）	单位频率跑分（Coremark/MHz）
STM32F072RBT6	片上 Flash	片上 Flash	48.000	1.8424
STM32F103C8T6	片上 Flash	片上 Flash	72.000	2.2104
STM32F407VGT6	片上 Flash	片上 Flash	168.00	2.8586
小胖达 RISC-V	片上 SRAM	DCache 缓存的外部 DDR3	100.00	3.1792
STM32H743	ICache 缓存的外部 Flash	片上 SRAM	400.00	5.0514

（1）STM32F072RBT6 是基于 ARM Cortex-M0 内核的，STM32F103C8T6 是基于 ARM Cortex-M3 内核的，STM32F407VGT6 是基于 ARM Cortex-M4F 内核的，STM32H743 是基于 ARM Cortex-M7F 内核的。

（2）小胖达 RISC-V 的运行频率是在 FPGA 原型上测得的，具体型号为 xc7z035ffg676-2。

（3）STM32 系列处理器的测试数据来源于嵌入式微处理器基准评测协会（EEMBC）官方，网址为 https://www.eembc.org/coremark/scores.php。

3.2.3 嵌入式程序测试

本文将小胖达 RISC-V 集成到了一个小型 SoC 上，构建嵌入式系统，从而体现该处理器核完成控制和交互类任务的能力。在这个 SoC 里，定时器组#1 包含 4 个定时器，用于产生 PWM 波以控制 4 个电机的速度；定时器组#2 也包含 4 个定时器，用于对 4 个电机产生的编码器脉冲进行计数，从而测量电机的实际速度；定时器组#3 只包含 1 个定时器，用于产生时钟基准；定时器组#4 也只包含 1 个定时器，用于捕获超声波测距模块产生的回波脉冲；通用异步收发器（UART）负责从蓝牙模块收发数据，以接收用户命令和发送状态信息；通用输入/输出（GPIO）用于产生超声波测距模块的触发脉冲、控制电机的转向、控制 LED 点阵、作数码管扫描显示。

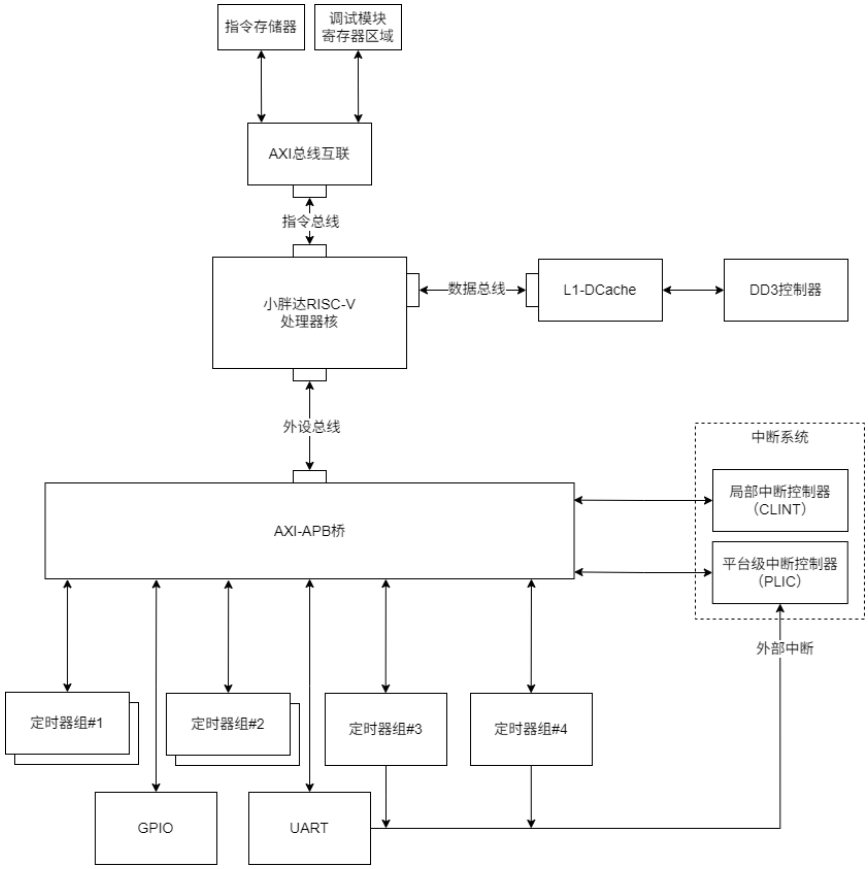


图 3-17 基于自研 RISC-V 的测试 Soc 框图

图 3-18 展示了该嵌入式系统的实拍图，这是一个遥控脉轮小车，通过蓝牙模块接收命令和传回状态，数码管上显示的是超声波测距的结果（以 mm 计）。

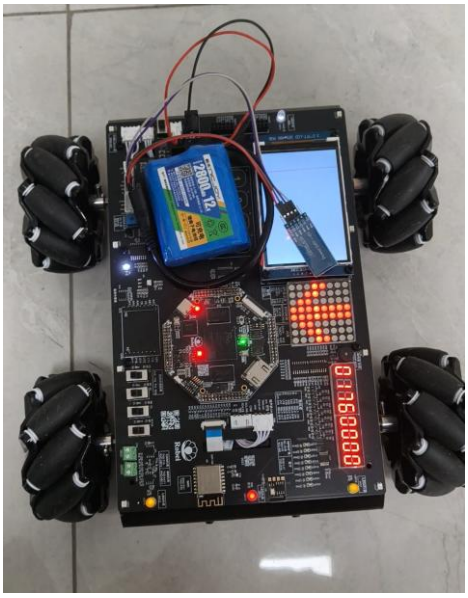


图 3-18 基于自研 RISC-V 的嵌入式系统测试实拍图

3.3 本章小结

本章完成了一种自研 RISC-V 处理器核的设计和验证工作。

首先介绍了自研处理器核的微架构，它是一个单发射、乱序、6 级流水线的 RV32IM 处理器。处理器采用了基于 BTB、RAS 和局部/全局历史的分支预测，它对有条件直接跳转指令（B 指令）的预测准确率可达 94.479%。处理器使用基于 ROB 的寄存器重命名来解决 WAW 和 WAR 数据相关性。处理器的发射策略是数据捕获的，它在发射之前有 1 级预取操作数的流水线。处理器带有组相连、写回、写分配的 L1-DCache，一般命中率可达 99%以上。指令的异常、中断和调试请求是在交付与退休阶段处理的。为了连接核内和外部的指令/数据总线，处理器核内实现了专门的总线互联单元（BIU）。除此之外，还介绍了处理器里两个重要的执行单元，也就是加载与存储单元（LSU）和除法器。

然后介绍了自研处理器核的测试，这包括基于 riscv-tests 的 ISA 兼容性测试、Coremark 跑分测试和嵌入式系统测试。除了不支持非对齐访存，处理器核能够通过 riscv-tests 里 rv32ui-p 和 rv32um-p 测试集里的其余测试。处理器核的跑分可达 3.1792Coremark/MHz，这相当于 STM32F4 系列 MCU 的性能。处理器核也可以很方便地集成到 SoC 上，搭建嵌入式系统，比如实现对遥控麦轮小车的控制。

可见，该自研处理器核设计完整、性能可观、方便使用，它将被集成到图像处理 SoC 上，用来完成控制和交互类任务。